BLM3522 BULUT BİLİŞİM DERSİ

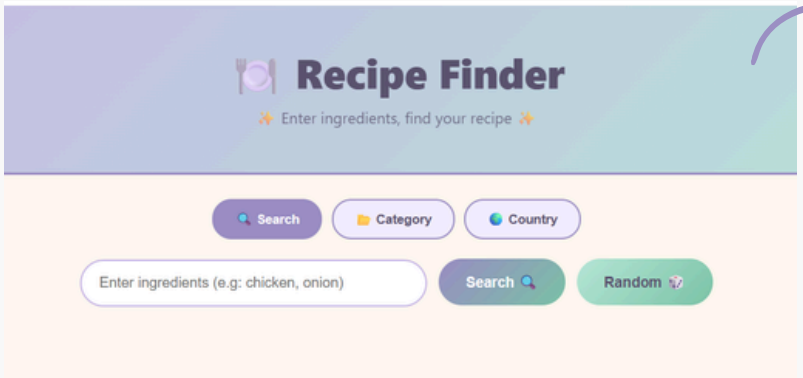
PROJE 1 RAPORU

Yemek Tarifi Bulma Uygulaması

Hazırlayan	**Yağmur Kalyoncu -23291413**
Ders	**BLM3522 Bulut Bilişim**
Proje	**Proje 1 - Çift Katmanlı Web Uygulaması**
GitHub	**https://github.com/Yagmurkalyoncu/yemek-tarifi-app**
Site	
Youtube	

1. Proje Amacı

Uygulama, kullanıcıların ellerindeki malzemelere göre yemek tarifleri arayabileceği, ülke ve kategoriye göre filtreleyebileceği, rastgele tarif alabileceği ve tariflerin Türkce çevirisine ulaşabilecegi kapsamlı bir platformdur. Veri kaynağı olarak ücretsiz TheMealDB API'si kullanılmıştır. Proje boyunca RESTful API mimarisi, React.js ile modern frontend geliştirme, AWS bulut platformunda deployment ve Git versiyon kontrol sistemi gibi yazılım geliştirme pratikleri uygulanmıştır.

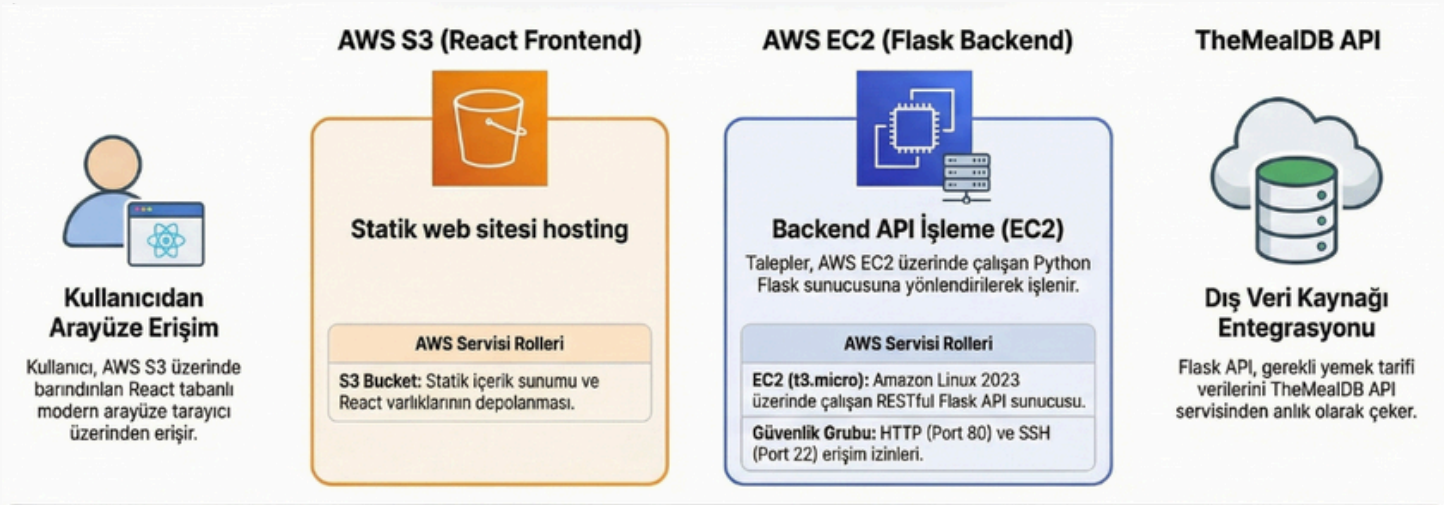


ÖZELLİKLER

- Malzemeye göre tarif arama
- Birden fazla malzemeyle aynı anda arama yapabilme
- Kategoriye göre filtreleme
- Ülkeye göre filtreleme
- Rastgele tarif önerisi
- Tarif detay sayfası (malzeme listesi + talimatlar)
- Google Translate entegrasyonu (Türkçe çeviri)
- TR/EN dil seçimi

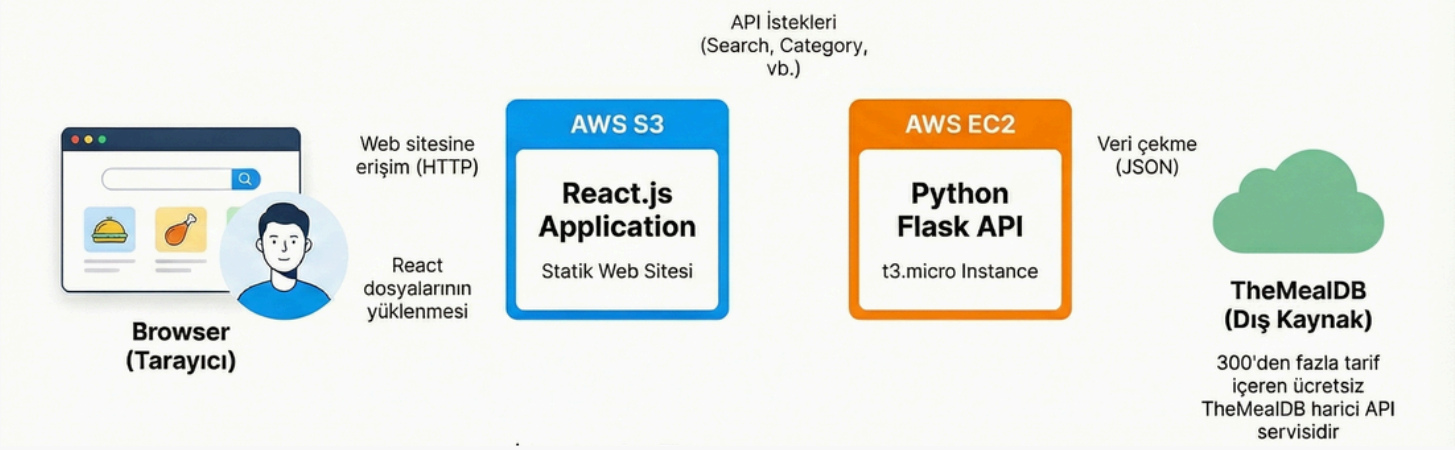
2.Kullanılan Teknolojiler

KATMAN	TEKNOLOJİ	AÇIKLAMA
Backend	Python + Flask	RESTful API geliştirme
Frontend	React.js	Kullanıcı Arayüzü
Harici API	TheMealDB	Yemek Tarifi Verisi
Bulut(Backend)	AWS EC2	Flask API Sunucusu
Bulut(Frontend)	AWS S3	Statik Web Sitesi
Versiyon kontrol	Git + Github	Kod Yönetimi



3.Sistem Mimarisi

Uygulama, modern web uygulamalarında yaygın olarak kullanılan çift katmanlı mimari üzerine inşaa edilmiştir. Bu mimari, frontend ve backend'in birbirinden bağımsız olarak geliştirilmesine ve konumlandırılmasına olanak tanımaktadır. Sunum katmanı AWS S3 üzerinde, işin mantığı ise AWS EC2 üzerinde çalıştırılmaktadır.



Kullanıcı tarayıcısı üzerinden S3'te olan React arayüzüne erişir. React uygulaması, kullanıcı isteklerini HTTP protokolu üzerinden EC2 üzerindeki Flask API'ye iletir. Flask, TheMealDB'den JSON formatında veri çekilmesini sağlar ve işlenmiş veriyi frontend'e geri gönderir. Bu mimari sayesinde her katman bağımsız olarak ölçeklendirilebilir ve yönetilebilir hale gelmektedir.

4.Backend Geliştirme -RESTful API

Bu aşamada uygulamanın "beyni" olan sunucu tarafı, Python ve Flask kullanılarak inşa edilmiştir. Sunucu, istemciden (Frontend) gelen talepleri karşılamak, dış servislerle (TheMealDB) haberleşmek ve veriyi uygun formatta sunmak üzere stateless (durumsuz) bir yapıda tasarlanmıştır.

4.1. Teknoloji Seçimi ve Ortam Kurulumu

Backend tarafında hafifliği ve esnekliği nedeniyle Flask tercih edilmiştir. Geliştirme sürecinde bağımlılıkların birbirine karışmaması için venv (virtual environment) kullanılarak izole bir çalışma alanı oluşturulmuştur.

Kullanılan Kütüphaneler:

- Flask: Web sunucusu ve rotaların yönetimi için.
- Flask-CORS: Frontend (S3) ve Backend (EC2) farklı domainlerde olduğu için oluşabilecek güvenlik (Cross-Origin Resource Sharing) engellerini aşmak için.
- Requests: Dış kaynaklı TheMealDB API'sine HTTP istekleri göndermek için.

```
from flask import Flask, jsonify, request
from flask_cors import CORS
import requests

# Uygulamayı başlat
app = Flask(__name__)
CORS(app)  # React frontend'in bağlanabilmesi için
```

4.2. RESTful Endpoint Tasarımı ve Veri Akışı

API tasarımı, standart REST prensiplerine uygun olarak GET metotları üzerinden şekillendirilmiştir. Her endpoint, belirli bir kaynağa (tarif, kategori, ülke) erişim sağlar ve sonucu JSON formatında döndürür.Toplam 6 endpoint, farklı kullanıcı ihtiyaçlarını karşılamak üzere tasarlanmıştır.

Endpoint	Teknik İşlevi
/api/saglik	Health Check: Sistemin çalışma durumunu doğrular
/api/search	Dinamik Arama : Malzeme bazlı sorguları işler
/api/detay/	Belirli bir tarifin tüm detaylarını getirir.
/api/kategori/	Sınıflandırma: Veri Gruplandırması sağlar.
/api/ulke/	Lokalizasyon: Coğrafi bazlı sınıflandırma sağlar.
/api/rastgele	Random veri sunulmasını sağlar

Önemli Kod Blokları ve Fonksiyonlar:

A. Uygulama Yapılandırması ve CORS:

Backend'in dış dünyaya açılması ve frontend ile konuşabilmesi için en kritik ayardır

```
backend > app.py > ...
from flask import Flask, jsonify, request
from flask_cors import CORS
import requests

# Uygulamayı başlat
app = Flask(__name__)
CORS(app)  # React frontend'in bağlanabilmesi için
```

B. Dinamik Arama ve Kesişim Mantığı (Search Endpoint):

Kullanıcının girdiği malzemeleri işleyen bu rota, sistemin en işlevsel parçasıdır.

```
# Malzemeye göre tarif ara
@app.route('/api/search', methods=['GET'])
def search():
    malzeme = request.args.get('malzeme', '')
    if not malzeme:
        return jsonify({'error': 'Malzeme giriniz'}), 400

    url = "https://www.themealdb.com/api/json/v1/1/filter.php?i=" + malzeme
```

C. Detay ve Filtreleme Servisleri:

Uygulamanın zenginliğini artıran kategori ve ülke bazlı filtreleme endpoint'leri

```
# Ülkeye göre tarif ara
@app.route('/api/ulke/<ulke>', methods=['GET'])
def ulke(ulke):
    url = "https://www.themealdb.com/api/json/v1/1/filter.php?a=" + ulke
    sonuc = requests.get(url).json()
    return jsonify(sonuc)
```

4.3 Veri Yönetimi ve API Entegrasyonu

Backend, kendi veritabanını tutmak yerine TheMealDB platformunu bir veri kaynağı (Data Source) olarak kullanır. Gelen istekler işlenerek şu adımlardan geçer:

- Frontend'den gelen malzeme ismi parametre olarak alınır.
- Python requests kütüphanesi ile dış API'ye sorgu atılır.
- Dönen karmaşık JSON verisi ayıklanarak, frontend'in en hızlı işleyebileceği sadeleştirilmiş JSON formatına dönüştürülür.

1.4. Deployment Hazırlığı (Host & Port Ayarları)

Yerel geliştirmede kullanılan 127.0.0.1 (localhost) adresi, AWS EC2 üzerinde dış dünyaya erişim veremeyeceği için, sunucu tüm IP adreslerini dinleyecek şekilde (0.0.0.0) ve standart HTTP portu olan 80 üzerinden yayına hazır hale getirilmiştir:

```
if __name__ == '__main__':
    app.run(debug=True, host='0.0.0.0', port=80)
```

4.Frontend Geliştirme

Uygulamanın arayüzü, modern ve bileşen tabanlı bir mimari sunan React.js kütüphanesi kullanılarak geliştirilmiştir. Kullanıcı dostu, hızlı ve etkileşimli bir deneyim sunmak hedeflenmiştir.